Heaven's Light is Our Guide



Rajshahi University of Engineering & Technology

Department of Electrical & Electronic Engineering

Project Report

"IoT-Based Smart DC Microgrid with Priority Load Shedding and Real-Time Mobile Alerts using ESP32"

Course No. : EEE 3202

Course Title : Capstone Project I

Submitted By:

1. Name : Md. Raiyan Mahmud
   Roll : 2201042
2. Name : Sameeha Tahseen
   Roll : 2201157

Submitted To:

Dr. Md. Sohel Rana

Professor

Dept. of EEE, RUET

1. Project in Brief

Title of the Project	:	IoT-Based Smart DC Microgrid with Priority Load Shedding and Real-Time Mobile Alerts using ESP32
Project Financed by	:	Self
Total Cost of Project Completion	:	BDT 3,661

2. Brief description of the project

a) Abstract

This project builds and tests a bench-scale DC microgrid that keeps three loads running from a 6 V, 4.5 Ah sealed lead-acid battery and reports what it is doing to a phone through Telegram. Power flows from a 12 V adapter through an XL4015 constant-current, constant-voltage charger to the battery, then through a 2 A fuse and an INA219 sensor to the load bus. An ESP32 reads the battery voltage and the total current, drives an LED bank, a fan and a DC motor (0.068 A, 0.14 A and 0.20 A, about 0.49 A in total with the electronics) and shows the readings on an OLED display. When the charger is absent and the battery voltage falls, the ESP32 cuts the Motor at 6.15 V, the Fan at 6.05 V and the LED at 5.85 V and restores them in the reverse order with hysteresis. An automatic overload limit of about 0.61 A, a 1.2 A short-circuit fault latch and an emergency button protect the system, and they keep working when WiFi is lost. Every event is sent to a phone through a Telegram bot. Nine functional tests passed on the bench, with the low-battery sequence verified by a simulated battery voltage. The prototype was built from market parts at a cost of about BDT 3,661.

b) Review of literature on the subject matter of the project and rationale behind the present initiative (within 1500 words):

A DC microgrid is a small power system in which sources, storage and loads share one DC bus. Many present-day loads, including LED lighting, fans, motor drives and electronics, are DC by nature, so a battery-backed DC bus can serve them without extra AC-DC conversion stages. DC microgrids have been reviewed widely, with attention to control, stability and protection [1], [2]. In Bangladesh the term load shedding is commonly used for scheduled utility outages. In microgrid engineering the term has a different meaning: the controller itself disconnects low-priority loads in a planned order so that the remaining loads and the battery survive a shortage [4], [5]. This project studies that second meaning on a small hardware prototype that students can build, measure and explain.

DC microgrid control. Dragicevic et al. [1] review control strategies and stabilization techniques for DC microgrids. They divide control into a local level, which uses only local measurements, and a coordinated level, which needs communication, and they distinguish decentralized, centralized and distributed coordination. Meng et al. [2] extend the review to hierarchical control, plug-and-play operation, stability, islanding detection and protection. Guerrero et al. [3] describe the hierarchical structure of primary (droop), secondary (restoration) and tertiary (power flow) control. These works concentrate on

converter-level control of larger systems with several sources. The local versus coordinated distinction is useful here: in this project the protection and the shedding decisions use only local measurements, and communication (WiFi and Telegram) is used for reporting and commands.

Load shedding. Mohammadi and Ajaei [4] propose an adaptive time-delay strategy that sheds loads in a pre-determined order of priority, keeps the DC bus voltage above a lower limit and avoids shedding more than necessary. Babagana et al. [5] combine a timer-based scheme with mixed integer linear programming to keep a DC bus above a critical voltage, and they note that communication-based schemes are complex, costly and vulnerable to communication failures. As described in their abstracts, both papers concentrate on the shedding principle (order, voltage limit and timing) and not on a complete low-cost prototype with measurement, display and phone alerts. The present project applies the same principle with a priority order (Motor, Fan, LED) and a local voltage threshold for each load. All protection is independent of the network, which avoids the weakness of communication-based schemes pointed out in [5].

IoT monitoring with Telegram. An ESP32 with a Telegram bot is a common low-cost platform. Izzudien et al. [6] monitor an off-grid solar plant with an INA219 for the battery and the load and switch a lamp remotely through Telegram, comparing sensor readings with a multimeter. Mulky et al. [7] describe an ESP32 load monitoring and control system with Telegram commands and an overload alert when the current passes a preset limit. These works show that Telegram is a practical interface for small power systems. In the abstracts reviewed, the emphasis is on measurement, remote control and notification. They do not describe a priority-ordered shedding sequence driven by battery voltage, a latched fault with an emergency stop, or protection that keeps working when the network is lost.

Battery state of charge. A shedding scheme ideally needs the battery state of charge (SoC). For lead-acid cells the terminal voltage falls as the cell discharges and also depends on the discharge current and the temperature [8], so voltage is only an approximate indicator under load. More accurate methods, such as coulomb counting and Kalman-filter-based estimation, are described in the battery management literature [9], [10], but they need extra sensing and tuning. In this project the measured voltage is corrected for the drop in the wires and the fuse (1.2 ohm times the current) and averaged over about 5 s, and SoC is shown only as an estimate between 5.85 V (0 %) and 6.35 V (100 %). A Kalman filter is left as future work.

Rationale. Among the works reviewed here, the literature gives strong foundations on DC microgrid control and on ordered load shedding, and many examples of ESP32 and Telegram monitoring. It gives fewer descriptions of a small, complete and low-cost system that combines a battery bus with a charger that can disappear, whole-bus current sensing that includes the controller's own consumption, priority-ordered shedding with hysteresis, local overload, fault and emergency protection, and phone alerts. This project fills that practical and educational gap with a prototype built from locally available parts for about BDT 3,661 and tested function by function. It is not claimed to be a new control method. Its value lies in an integrated, documented and reproducible design.

c) Objectives and aims of the project (within 150-200 words)

I. To build a bench-scale DC microgrid in which a 6 V, 4.5 Ah sealed lead-acid battery, kept charged from a 12 V adapter through a CC/CV charger, supplies three loads: an LED bank, a fan and a DC motor.

II. To measure the battery voltage and the total load current in real time with an INA219 sensor and show them on an OLED display.

III. To shed loads in a fixed priority order (Motor, Fan, LED) as the battery voltage falls, and to restore them in the reverse order with hysteresis.

IV. To protect the system with an automatic overload limit (about 0.61 A), a 1.2 A short-circuit fault latch and an emergency stop button, all working without WiFi.

V. To send every event to a phone as a Telegram message and to accept control commands from the owner's chat only.

VI. To complete the prototype from locally available parts at a total cost below BDT 4,000 and to verify each function by test.

d) Experimental work/Methodology adopted in the project (detailed as far as possible)

System model and description

The prototype is a single-bus DC microgrid (Fig. 1 and Fig. 2). The battery is the energy store and the bus. The 12 V adapter plays the role of the utility grid: when it is connected, the XL4015 charger (set to 6.9 V with a 0.5 A limit) charges the battery, and when it is removed the battery alone supplies the loads. Everything the battery supplies, including the controller, passes through a 2 A fuse and the INA219 sensor, so one sensor measures the total current. The charger is connected ahead of the sensor, so the charging current is not measured. The MP1584 buck module takes its input from the load bus and gives a 4.96 V rail for the ESP32 and the buzzer. The adapter reaches the ESP32 only through a 100 kohm and 22 kohm divider on pin D34, which gives about 2.16 V (12 V × 22/122) when the adapter is present and 0 V when it is absent. The ESP32 reads the INA219 over I2C, switches the three loads, drives the buzzer and sends alerts through WiFi.

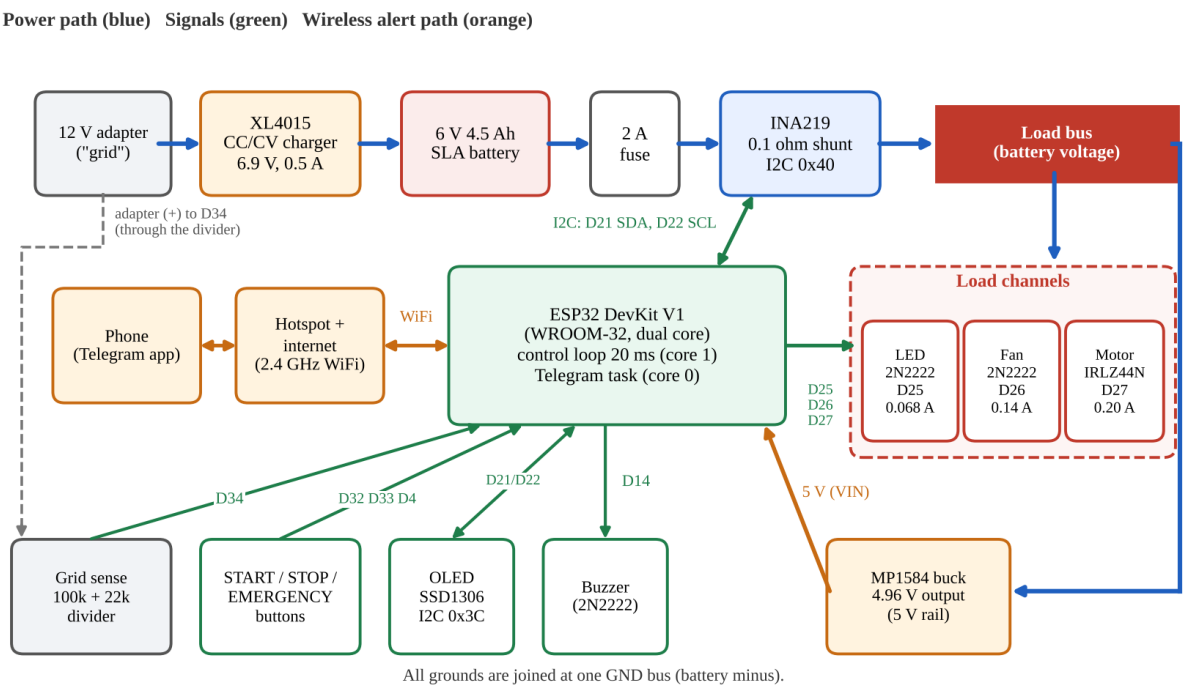


Fig. 1. Block diagram of the system. Power path in blue, signals in green, wireless alert path in orange.

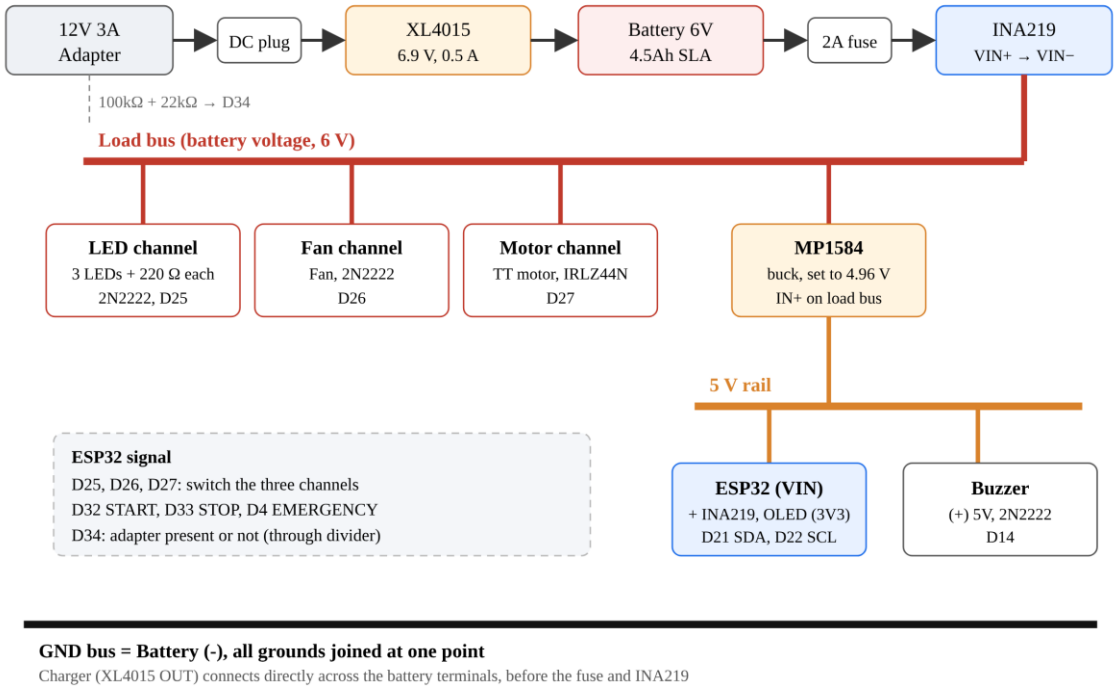


Fig. 2. Power path of the prototype. The charger is connected across the battery, ahead of the fuse and the INA219. The MP1584 takes its input from the load bus.

System parameters and hardware design

Table 1 lists the main parts and their roles, and Table 2 gives the pin map (D numbers are the labels printed on the ESP32 board).

Table 1. Main components and key values.

Part	Role	Key value
ESP32 DevKit V1 (WROOM-32)	Controller, WiFi, Telegram	3.3 V logic, two cores, 240 MHz
INA219 module	Current and bus voltage sensor	0.1 ohm shunt, I2C 0x40, 32 V / 2 A calibration
SSD1306 OLED, 0.96 in	Local display	128 x 64, I2C 0x3C
6 V 4.5 Ah SLA battery	Energy storage and load bus	about 6.4 V full, at rest
XL4015 CC/CV module	Charger from the adapter	set to 6.9 V, 0.5 A limit
MP1584 buck module	5 V rail for ESP32 and buzzer	set to 4.96 V, input from the load bus
IRLZ44N MOSFET	Motor switch	100 ohm gate resistor, 10 kohm pull-down
2N2222 transistors	LED, fan and buzzer switches	base resistor, 10 kohm pull-down
1N5819 Schottky diode	Motor flyback protection	cathode on the load bus
Loads	LED bank (3 LEDs, 220 ohm each), DC fan, TT gear motor	0.068 A, 0.14 A, 0.20 A
2 A fuse	Series protection	between the battery node and the INA219

Table 2. Pin map of the ESP32.

Pin	Function	Connected to
D21 / D22	I2C SDA / SCL	INA219 and OLED
D25	LED channel	2N2222 base through 1 kohm
D26	Fan channel	2N2222 base through a resistor
D27	Motor channel	IRLZ44N gate through 100 ohm
D14	Buzzer	2N2222 base through 1 kohm
D32 / D33 / D4	START / STOP / EMERGENCY	push buttons to GND (internal pull-ups)
D34	Grid sense (ADC input)	adapter plus through 100 kohm, 22 kohm to GND, 0.1 uF filter

The LED and fan channels use 2N2222 transistors as low-side switches: the emitter goes to GND, the load sits between the load bus and the collector, and a base resistor connects the ESP32 pin. The motor channel uses a logic-level IRLZ44N MOSFET, because the ESP32 can drive the gate only to 3.3 V, with a 1N5819 flyback diode and a 0.1 uF capacitor across the motor (Fig. 3). The charger (6.9 V, 0.5 A) and the buck module (4.96 V) were set with a multimeter before the battery and the controller were connected.

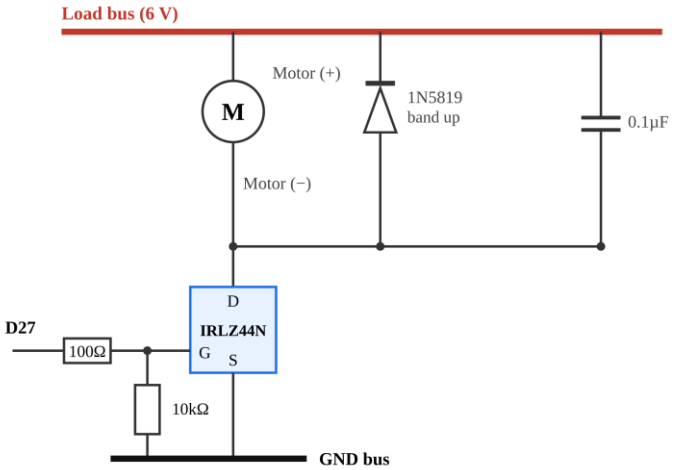


Fig. 3. Motor channel. The 1N5819 diode and the 0.1 uF capacitor sit across the motor, and the IRLZ44N switches the motor to the GND bus.

Algorithm, flow chart and calibration equations

The control loop runs every 20 ms on core 1 and applies four rules in a fixed priority, shown in Fig. 4. Emergency has the highest priority, then Fault, then Overload, then Low battery. After the rules, loads that were cut may be restored one by one. Table 3 summarizes the rules and Fig. 5 shows the voltage thresholds.

Table 3. Protection rules and their return conditions.

Rule	Trigger	Action	Return
Emergency	EMERGENCY button	All loads off, buzzer, latched	Release the button, then /reset or START
Fault	Current above 1.2 A for 0.1 s	All loads off, buzzer, latched	/reset or START
Overload	Average current above the limit for 0.4 s	Cut the lowest-priority load that is on (Motor, Fan, LED)	After at least 20 s, if there is room
Low battery	Charger absent and battery below 6.15 V (Motor), 6.05 V (Fan), 5.85 V (LED)	Cut that load	Above 6.23 V, 6.13 V, 5.95 V, one load every 1.5 s

The calibrated battery voltage, the automatic overload limit and the displayed state of charge are computed as follows:

$$V_batt = (V_bus + V_shunt) \times 0.990 + I \times 1.2\ \Omega \quad (1)$$

$$I_LIMIT = I_idle + I_LED + I_FAN + I_MOT + 0.12\ A = 0.08 + 0.068 + 0.14 + 0.20 + 0.12 \approx 0.61\ A \quad (2)$$

$$SoC\ (\%) = (V_batt - 5.85) / (6.35 - 5.85) \times 100,\ limited\ to\ 0\ to\ 100 \quad (3)$$

In Eq. (1), 0.990 is the voltage calibration factor found against a multimeter, and 1.2 Ω is the estimated resistance of the wires and fuse, added back so that a load turning on is not mistaken for a flat battery. In Eq. (2) the idle current of the ESP32 and the sensors (0.08 A on battery) is measured at boot with all loads off. The limit therefore sits above normal operation (0.49 A) and trips only on an abnormal current such as a stalled motor. On USB power the idle current is about 0.01 A and the limit is about 0.54 A.

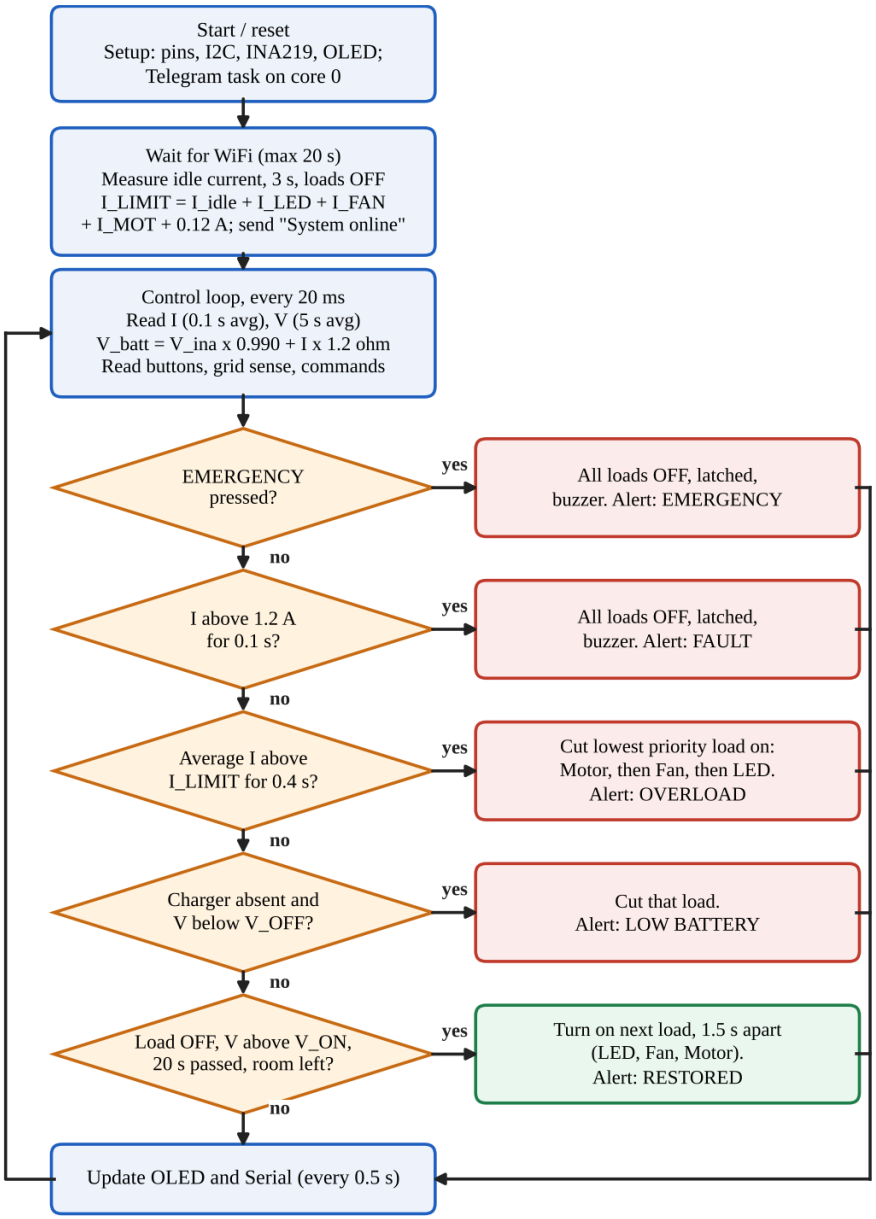


Fig. 4. Flow chart of the control loop.

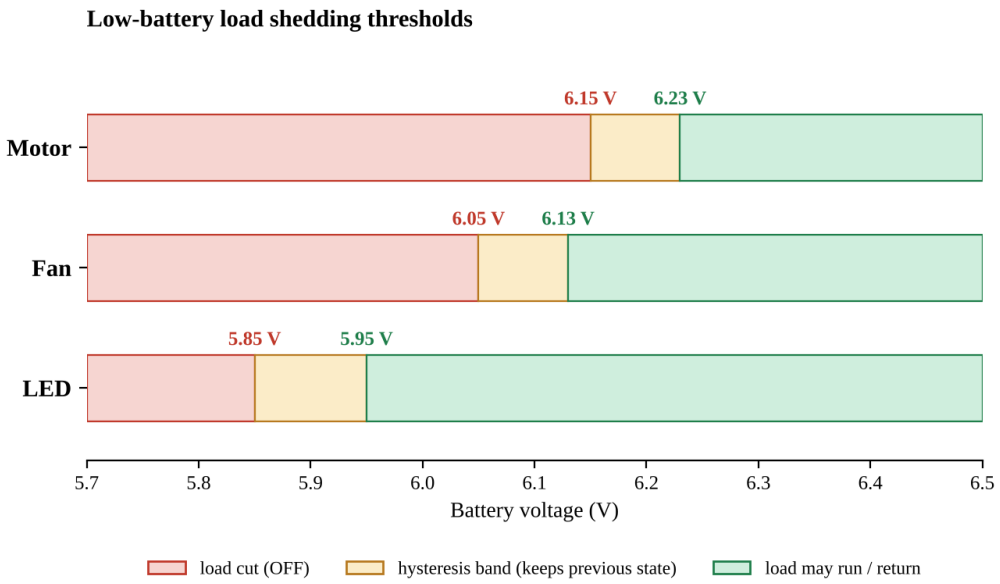


Fig. 5. Cut-off and return voltages of the three loads. Between the cut-off and the return voltage a load keeps its previous state (hysteresis).

Measurement details

- Current is averaged over about 0.1 s and voltage over about 5 s. Samples outside 4 V to 7.6 V are discarded, and negative current is treated as noise.
- Current is ignored for 0.4 s after a load turns on, to skip the motor start-up surge. Overload is not judged during short WiFi bursts, and the limit is raised by 0.05 A when WiFi is down, because the radio then scans continuously.
- If the INA219 returns exactly 0 A and 0 V, it is re-initialized after 0.2 s, and its calibration is refreshed every 2 s [12].
- Buttons are debounced for 30 ms. The grid signal is debounced for 1.5 s (ADC above 1500 means present, below 1000 means absent). While the charger is present, battery-based cutting is disabled, but overload and fault protection stay on.

Telegram interface and task structure

The control loop runs on core 1. The Telegram library runs as a separate task on core 0 with the lowest priority and receives events through a queue of eight messages, so protection never waits for the network. The task polls the bot every 5 s, retries a failed message at most three times, and accepts commands only from the owner's chat ID [13], [14]. Low priority and the retry limit matter, because the library waits in busy loops and a long wait can starve the idle task and trigger the ESP32 watchdog [15]. Events sent to the phone are: system online, overload, fault, emergency, low battery, restored, reset, grid lost and grid back. Table 4 lists the commands.

Table 4. Telegram commands (owner's chat only).

Command	Effect
/status	State, grid, battery voltage and percent, current, limit, which loads are on
/start, /stop, /reset	Request all loads and clear a latch; all loads off; clear a fault or emergency latch
/sim 6.00, /sim off	Pretend the battery is at that voltage (test mode), or return to the real reading
/trip 0.30, /trip off	Temporarily lower the fault threshold, for demonstration
/limit 0.40, /limit off	Temporarily lower the overload limit, for demonstration

Description of the prototype

The prototype is built on two solderless breadboards (the ESP32 on its own board) with jumper wires, mounted on a flat board next to the battery (Fig. 6). The adapter, the XL4015 charger, the MP1584 buck module, the INA219, the OLED, the LED bank, the small fan and the yellow TT gear motor are all visible. The wiring path adds about 1.2 Ω, which is compensated in firmware. Upload and Serial monitoring use the USB cable, and all tests and demonstrations run from the battery with the USB cable removed.

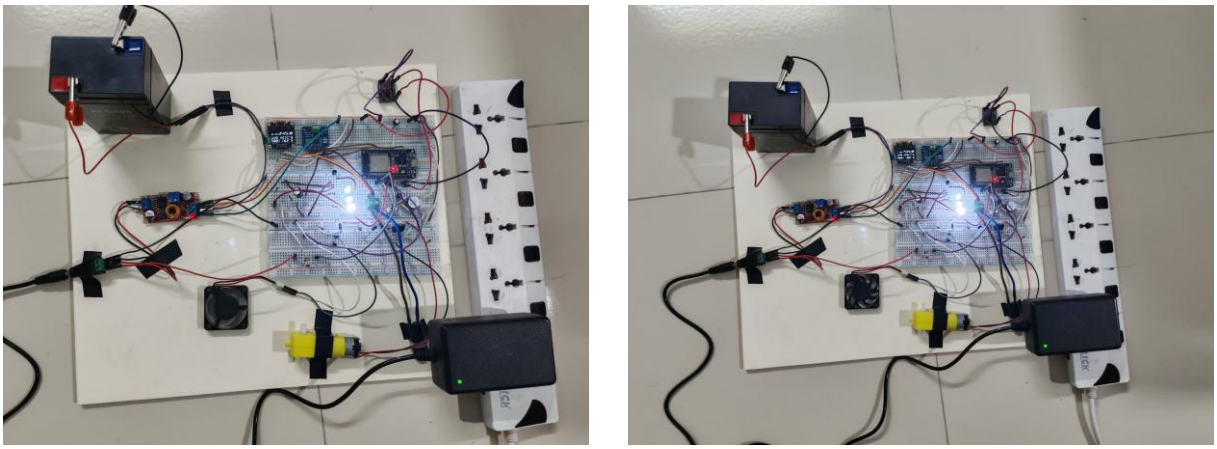


Fig. 6. The prototype. Left: all three loads running from the battery, with the adapter connected (OLED reads 6.43 V, 100 %, 0.519 A, source BAT). Right: start-up sequence, with the LED on first and the Fan and Motor still off.

e) Results and Discussions

All tests were carried out on 28 October 2026 with the system running from the battery only (USB removed), a 2.4 GHz phone hotspot for WiFi, and the adapter unplugged unless stated. Each load current in Table 5 is the rise in battery current when that load is switched on with the others off.

Table 5. Calibration and measured values.

Quantity	Value	How obtained
Voltage calibration factor	0.990	INA219 reading compared with a multimeter
Wiring and fuse path resistance	1.2 ohm	Estimated from the voltage sag under load, compensated in firmware
LED channel current	0.068 A	Load on minus all off
Fan channel current	0.14 A	Load on minus all off
Motor channel current	0.20 A	Load on minus all off
ESP32 and sensors, on battery	0.08 A	Idle current measured at boot, loads off
All three loads running	about 0.49 A	Measured during the overload test

Quantity	Value	How obtained
Same, with WiFi lost	0.56 A	Measured in the WiFi-off test, radio scanning
Automatic overload limit	0.61 A	Idle + loads + 0.12 A, set at boot
Fault trip	1.2 A for 0.1 s	Fixed threshold
Charger setting	6.9 V, 0.5 A limit	Set with a multimeter before connecting the battery
5 V rail	4.96 V	MP1584 trimmer, set with a multimeter
Grid-sense voltage on D34	2.14 to 2.18 V (adapter on), 0 V (off)	100 kohm and 22 kohm divider; ADC thresholds 1500 (on) and 1000 (off)

The load currents add up as expected: 0.08 + 0.068 + 0.14 + 0.20 = 0.488 A, which matches the 0.49 A measured with all loads running (Fig. 7). The 0.61 A limit leaves about 0.12 A (roughly 25 %) of headroom over normal operation. With WiFi lost the measured current rises to 0.56 A because the radio scans continuously, and the limit is raised by 0.05 A in that state so that normal operation does not trip.

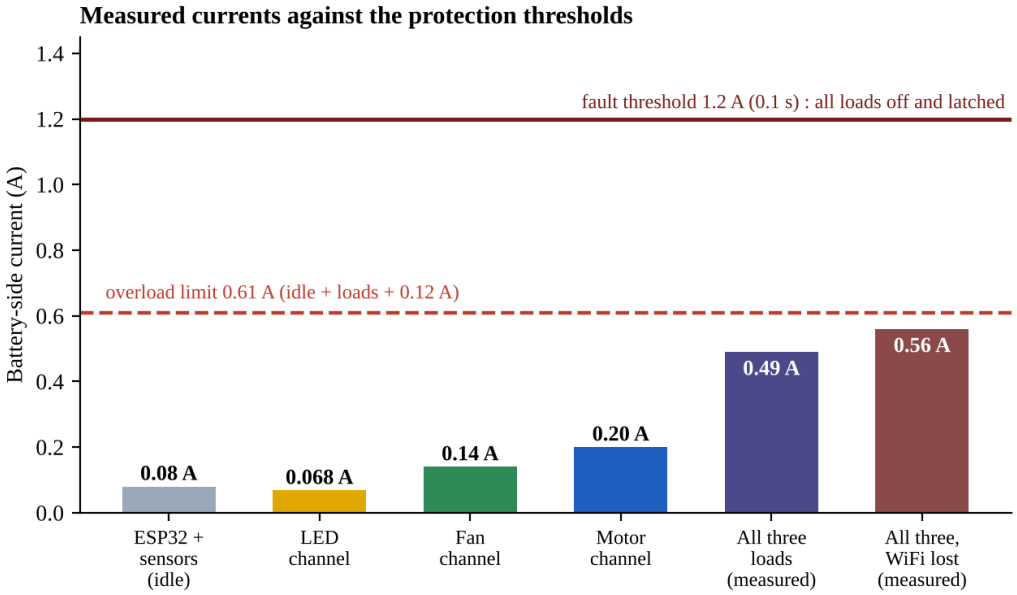


Fig. 7. Measured currents against the overload limit and the fault threshold.

Test procedure and results

Nine functional tests passed (Table 6). Two load policies were used during testing. In the first firmware (policy A) the Motor started only on START and the overload limit lay between LED + Fan and all three loads. In the final firmware (policy B) all three loads start at boot and the limit sits above them, so that they run together and trip only on an abnormal current. Tests 4, 7, 8 and 9 were run with policy A. The protection logic they exercise is unchanged, and they are to be repeated on the final firmware.

Table 6. Functional tests (28 October 2026).

#	Test	Action	Observed	Result
1	Power-up on battery	Connect battery, hotspot on	"System online" on Telegram. Idle 0.08 A. Limit 0.39 A (A), 0.61 A (B)	Pass
2	Boot sequence	Wait 10 s, touch nothing	A: LED then Fan on, nothing cut. B: LED, Fan, Motor on, about 0.49 A, nothing cut	Pass

#	Test	Action	Observed	Result
3	Overload	A: press START. B: send /limit 0.40	A: 0.49 A above 0.39 A, Motor off in about 1 s, buzzer. B: Motor cut by the lowered limit, back after /limit off	Pass
4	Short-circuit fault	/trip 0.30, then /trip off, /reset	Fault at 0.37 A, all loads off and latched. After reset LED then Fan returned (A)	Pass
5	Low battery (simulated)	/sim 6.10, 6.00, 5.80 (B)	Motor off at 6.10 V (49 %), Fan off at 6.00 V (30 %), LED off at 5.80 V (0 %)	Pass
6	Recovery	/sim 6.30, then /sim off	LED, Fan, Motor returned one by one, no chattering	Pass
7	Emergency stop	Hold EMERGENCY, send /reset, release, /reset	All loads off, buzzer. Reset refused while held, accepted after release (A)	Pass
8	No WiFi	Hotspot off, press START	Motor cut in about 1 s at 0.56 A, OLED showed WiFi --, alert delivered after reconnect (A)	Pass
9	Grid detection	Plug the adapter, wait, unplug	Grid back with OLED CHG, then grid lost with OLED BAT	Pass

Test 8 failed on its first run: the Motor was not cut, because the firmware skipped the overload check whenever the network task was busy, and with no WiFi it was always busy. The check was corrected and the test repeated successfully. The low-battery sequence (tests 5 and 6) was verified with a simulated battery voltage (Table 7). A full real discharge run is still to be done.

Table 7. Low-battery response with the simulated battery voltage (final firmware).

Simulated voltage	SoC shown	LED	Fan	Motor	Telegram alert
Real, 6.35 V	100 %	ON	ON	ON	[RESTORED] Motor ON, after /limit off
/sim 6.10	49 %	ON	ON	OFF	[LOW BATTERY] 6.10 V. Motor OFF
/sim 6.00	30 %	ON	OFF	OFF	[LOW BATTERY] 6.00 V. Fan OFF
/sim 5.80	0 %	OFF	OFF	OFF	[LOW BATTERY] 5.80 V. LED OFF
/sim 6.30	90 %	ON	ON	ON	[RESTORED] LED, Fan, Motor one by one

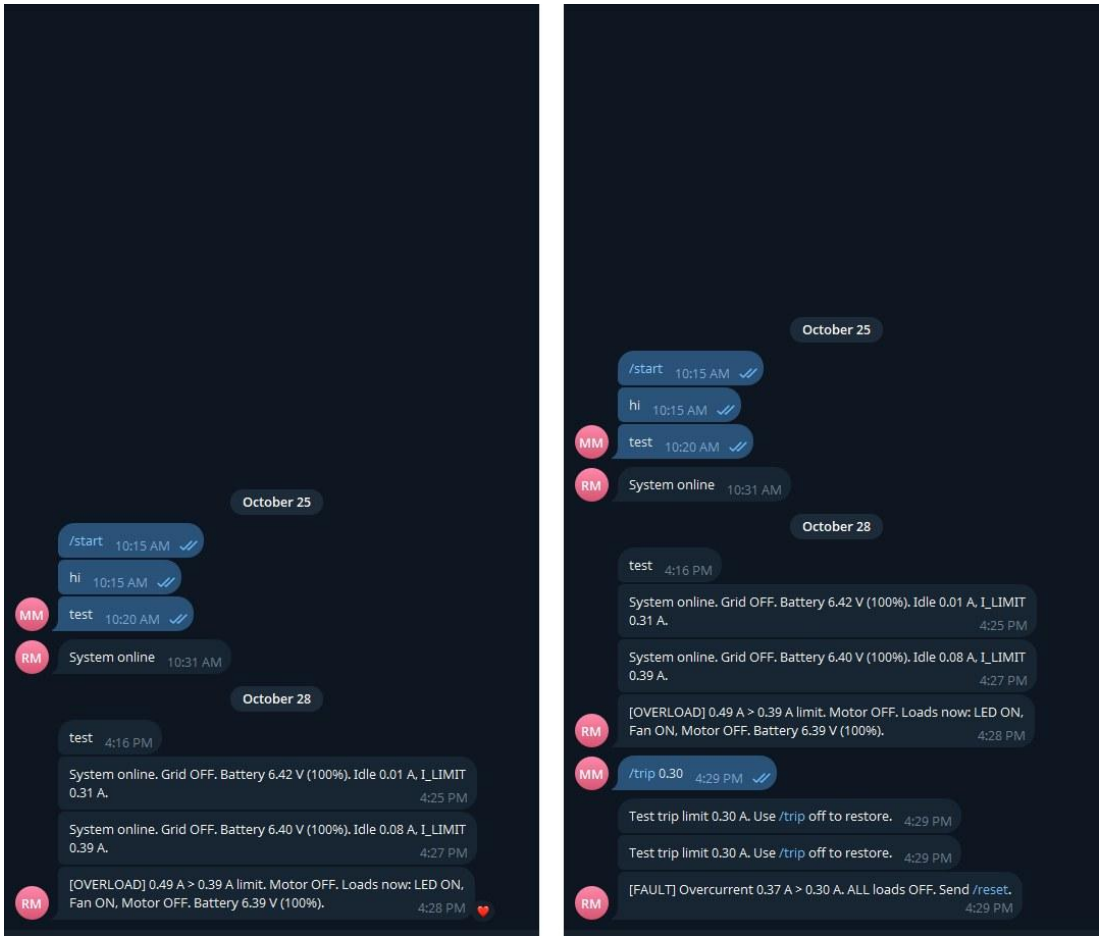


Fig. 8. Telegram alerts. Left: first power-up messages ("System online", USB and then battery power) and the overload alert, 0.49 A against a 0.39 A limit, Motor cut (first load policy). Right: fault test with /trip 0.30, overcurrent 0.37 A, all loads off.

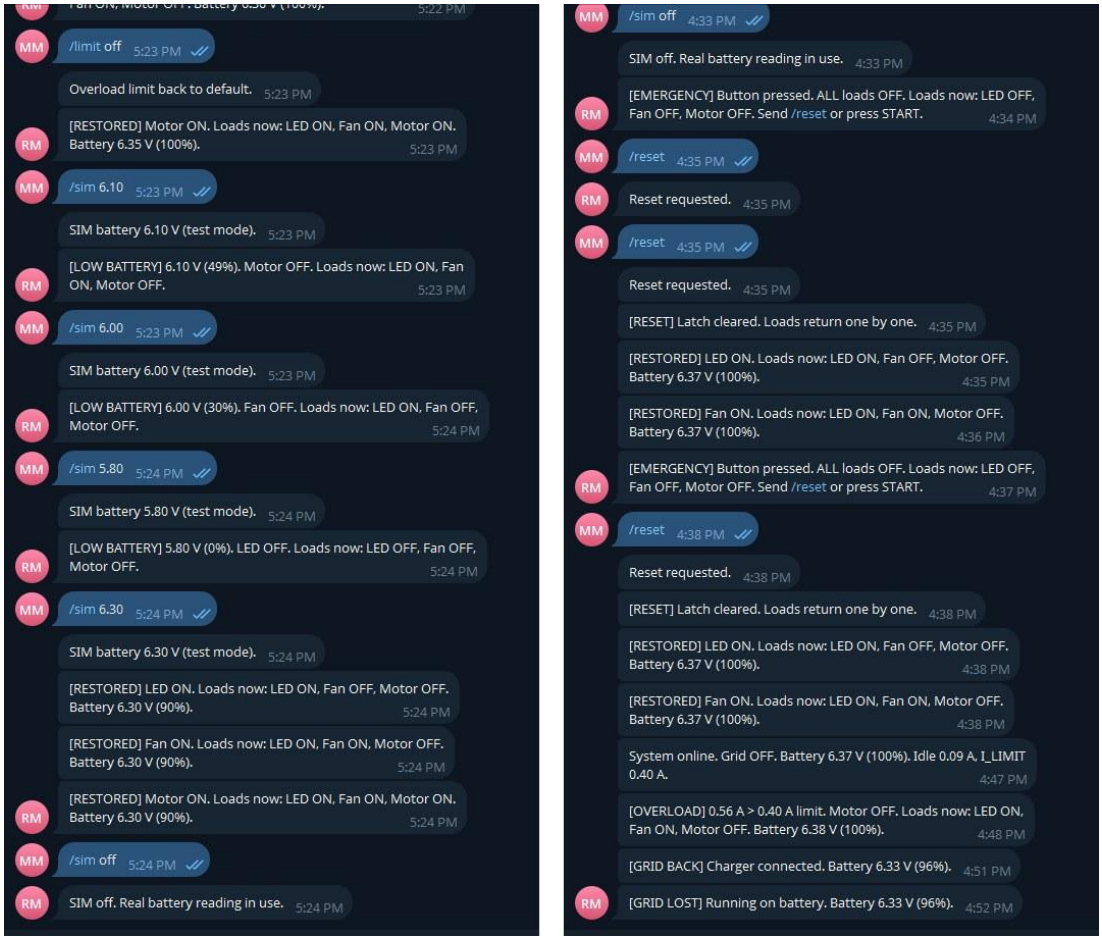


Fig. 9. Telegram alerts. Left: final policy. /limit off restores the Motor, then /sim 6.10, 6.00 and 5.80 cut Motor, Fan and LED in order and /sim 6.30 brings them back. Right: emergency stop and reset, overload at 0.56 A with no WiFi (delivered after reconnecting), then grid back and grid lost.

Discussion

The protection works on the bench. Testing found four firmware problems that a short demonstration would have hidden:

- **Repeated reboots.** The Telegram library waits in busy loops, and with a wrong chat ID it retried for 8 s. That starved the idle task on core 0, and the watchdog rebooted the ESP32 about every 16 s. Fixes: lower task priority, at most three send attempts, correct chat ID.
- **No overload protection without WiFi.** The check was skipped while the network task was busy. Fixed, and verified in test 8.
- **INA219 reading 0 A.** It briefly returned zeros just before a reboot. The firmware now re-initializes it and refreshes its calibration every 2 s.
- **Duplicate alerts on a slow connection**, because the library resends when a reply is late. This is harmless and can be reduced by raising the reply wait time.

The main limitations are:

- State of charge is estimated from voltage only, with a fixed wiring-drop correction. A real discharge run is still to be done, and the low-battery sequence was verified with a simulated voltage.
- Charging current is not measured, because the charger sits upstream of the INA219.

- The overload limit follows the measured load currents, so replacing a load means measuring it again. A stalled motor may exceed 1.2 A and trigger the fault instead of the overload.
- The wiring path is 1.2 Ω on breadboards, and there is one 2 A fuse, on the battery side only. Soldered joints and a second fuse on the adapter side would improve the system.
- Alerts need a 2.4 GHz WiFi network and internet. The protection does not.

Safety: the board carries 6 V DC only. Mains voltage stays inside the adapter, and the battery is connected last, through the fuse.

f) Financial Summary

Sl. No.	Item Description		Cost	
	Name of the Item	Quantity	Unit Price (BDT)	Total Cost (BDT)
1	ESP32 DevKit V1	1	640	640
2	INA219 current sensor module	1	243	243
3	OLED display	1	340	340
4	MP1584 buck module	1	80	80
5	12 V adapter and XL4015 CC/CV module	1 set	450	450
6	Sealed lead-acid battery	1	680	680
7	IRLZ44N MOSFET	3	80	240
8	2N2222 transistor	2	15	30
9	White LED with 220 ohm resistor	3 sets	7	21
10	DC fan	1	120	120
11	TT gear motor	1	80	80
12	Fuse with holder	1	45	45
13	Schottky diode 1N5819	2	15	30
14	Active buzzer	1	30	30
15	Push button	3	8	24
16	Resistors and capacitors	1 set	70	70
17	Screw terminal	4	12	48
18	Breadboard	1	200	200
19	Jumper wires	1 set	150	150
20	Wire, battery tab connectors, tape	1 set	140	140
			Grand Total	**3,661**

.

g) Project Timeframe:

The tentative week-wise plan for the work is shown below.

Task	W1	W2	W3	W4	W5	W6	W7	W8
Literature review and system design	■	■						
Component procurement		■	■					
Breadboard assembly and module testing			■	■				
Firmware development (measurement, shedding, protection)				■	■	■		
Telegram integration and WiFi-loss handling					■	■		
Calibration and system testing						■	■	
Performance evaluation and result analysis							■	■
Report writing and demonstration							■	■

h) Addressing Program Outcomes (POs)

The project addresses PO(j) and PO(k) as follows.

POs	Justification
PO(j): Communication	The project is communicated at two levels. For engineers, it is documented in this report and in a proposal with a block diagram, power-path and channel circuits, a flow chart, parameter tables and a test table. For the operator, who need not be an engineer, the system itself communicates: the OLED shows voltage, current, state and WiFi status, a buzzer sounds on every cut, and every event reaches the phone as a plain-language Telegram message such as "[OVERLOAD] 0.49 A above the 0.39 A limit. Motor OFF". Commands can be sent back in a few words. The work was presented through photographs, screenshots and a short demonstration video.
PO(k): Project Management and Finance	The work was planned over eight weeks (Gantt chart), from literature review and procurement to module-wise testing, firmware, calibration and reporting. The budget was kept to about BDT 3,661 by deliberate choices: a 6 V battery instead of 12 V, an MP1584 module because an LM2596 cannot regulate 5 V from a 6 V input, a voltage-based charge estimate instead of a coulomb-counting IC, and parts that are available in local shops or online. Risk was managed with a 2 A fuse, a staged build with every module tested on its own, and a charger and buck module set with a multimeter before the battery was connected. The team shared design, assembly, testing and documentation.

i) Addressing Complex Engineering Problems (P)

The project addresses P1, P2 and P4 as follows.

Attributes	Justification
P1 (Depth of knowledge)	The project needs knowledge from several sub-disciplines at once: DC power and the voltage behaviour of a lead-acid battery, constant-current and constant-voltage charging, buck conversion, transistor and

	MOSFET switching with a flyback diode, shunt-based current sensing and its calibration, I2C communication, dual-core real-time firmware on the ESP32 with task priorities and a watchdog, and HTTPS messaging through the Telegram Bot API.
P2 (Range of conflicting requirements)	The overload limit must sit above the normal 0.49 A load so that normal operation never trips, yet low enough to catch a stalled motor long before the 1.2 A fault level. Protection must be fast (0.4 s and 0.1 s) but must ignore the motor start-up surge and WiFi current bursts. The cut and return voltages must be far enough apart to avoid chattering without wasting battery capacity. Alerts must be reliable, but protection must not depend on the network. All this had to be achieved with a budget of about BDT 3,661.
P4 (Familiarity of issues)	The work met practical issues typical of real embedded power systems: 1.2 Ω of wiring and fuse resistance lowering the measured voltage under load, motor noise on the readings, a charging voltage of about 7.3 V that is too high for a 5 V fan, a transistor whose emitter was wired to the wrong rail, a logic-level MOSFET needed for a 3.3 V gate drive, the INA219 returning zeros, watchdog reboots caused by the Telegram library, and an overload test that failed without WiFi. Each was found by measurement and corrected.

j) Addressing Complex Engineering Activities (A)

The project addresses A1 and A2 as follows.

Attributes	**Justification**
A1 (Range of resources)	The project used a range of resources: two team members, a budget of about BDT 3,661, electronic components from local and online suppliers, a multimeter for calibration, software tools (Arduino IDE, the ESP32 board package, open-source libraries and a Telegram bot), published papers and component datasheets, and a limited time frame of a few weeks.
A2 (Level of interaction)	The work required interaction between several subsystems (power, sensing, switching, firmware and network) and between people: team members shared hardware and software tasks, and components had to be sourced from suppliers. The finished system also interacts with its user, who receives alerts and sends commands from a phone.

k) Conclusion and Future scope

- **Objectives.** The prototype met its objectives on the bench. It runs the LED, fan and motor from a 6 V battery (objective I), measures voltage and current and shows them on the OLED (II), cuts and restores the loads in priority order with hysteresis (III), protects against overload, short circuit and emergency without WiFi (IV), reports every event to a phone and accepts commands from the owner only (V), and was built for about BDT 3,661 (VI). Nine functional tests passed on 28 October 2026.
- **Open items.** The low-battery sequence was verified with a simulated battery voltage, so one real discharge run from full to the first load cut, with a time-voltage-current log, is still to be done. Tests 4, 7, 8 and 9 are to be repeated on the final firmware.

- **Future scope.** (a) A Kalman filter and coulomb counting for a better state-of-charge estimate [10], and load prediction for preventive shedding. (b) Current sensing on the charger side. (c) A temperature sensor and over-temperature protection. (d) A printed circuit board in place of the breadboards, with soldered joints. (e) A 12 V version with a solar source, and a web dashboard in addition to Telegram.

l) References

[1] T. Dragicevic, X. Lu, J. C. Vasquez, and J. M. Guerrero, "DC microgrids-Part I: A review of control strategies and stabilization techniques," IEEE Trans. Power Electron., vol. 31, no. 7, pp. 4876-4891, 2016, doi: 10.1109/TPEL.2015.2478859.

[2] L. Meng et al., "Review on control of DC microgrids and multiple microgrid clusters," IEEE J. Emerg. Sel. Topics Power Electron., vol. 5, no. 3, pp. 928-948, 2017.

[3] J. M. Guerrero, J. C. Vasquez, J. Matas, L. G. de Vicuna, and M. Castilla, "Hierarchical control of droop-controlled AC and DC microgrids-A general approach toward standardization," IEEE Trans. Ind. Electron., vol. 58, no. 1, pp. 158-172, 2011, doi: 10.1109/TIE.2010.2066534.

[4] J. Mohammadi and F. B. Ajaei, "Adaptive time delay strategy for reliable load shedding in the direct-current microgrid," IEEE Access, vol. 8, pp. 114509-114518, 2020, doi: 10.1109/ACCESS.2020.3002935.

[5] A. Babagana, I. A. Jimoh, Y. Seferi, and G. Burt, "A timer and mixed integer linear programming load shedding scheme for resilient DC microgrids," IEEE Access, vol. 13, pp. 6632-6642, 2025, doi: 10.1109/ACCESS.2025.3525975.

[6] M. A. Izzudien, M. Widyartono, A. C. Hermawan, and N. V. Laksmi B, "Perancangan sistem pengendali lampu jarak jauh dan monitoring kinerja PLTS off-grid berbasis Internet of Things (IoT) menggunakan ESP32 dan Telegram," [Online]. Available: https://ejournal.unesa.ac.id/index.php/jket/article/view/81802

[7] I. Mulky, Mutmainnah, and A. I. Asry, "Design and development of a remote electric load control system based on the Internet of Things (IoT) for energy savings," 2025, doi: 10.61844/jeat.v4i1.1202. [Online]. Available: https://journal.atim.ac.id/index.php/jeat/article/view/1202

[8] D. Linden and T. B. Reddy, Eds., Handbook of Batteries, 3rd ed. New York, NY, USA: McGraw-Hill, 2002.

[9] G. L. Plett, Battery Management Systems, Volume II: Equivalent-Circuit Methods. Norwood, MA, USA: Artech House, 2015.

[10] R. E. Kalman, "A new approach to linear filtering and prediction problems," Trans. ASME J. Basic Eng., vol. 82, ser. D, pp. 35-45, 1960.

[11] Espressif Systems, "ESP32 Series Datasheet," Shanghai, China. [Online]. Available: https://www.espressif.com/en/support/documents/technical-documents

[12] Texas Instruments, "INA219 zero-drift, bidirectional current/power monitor with I2C interface," Datasheet SBOS448. [Online]. Available: https://www.ti.com/product/INA219

[13] Telegram, "Telegram Bot API." [Online]. Available: https://core.telegram.org/bots/api

[14] B. Lough, "Universal-Arduino-Telegram-Bot," GitHub repository. [Online]. Available: https://github.com/witnessmenow/Universal-Arduino-Telegram-Bot